

OS LAB WEEK 7

1) Safety Algorithm

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    int i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m];
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < m; j++) {
```

```
            scanf("%d", &alloc[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Enter Maximum Demand Matrix:\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < m; j++) {
```

```
            scanf("%d", &max[i][j]);
```

```
    }  
}
```

```
printf("Enter Available Resources:\n");  
for(i = 0; i < m; i++) {  
    scanf("%d", &avail[i]);  
}
```

```
for(i = 0; i < n; i++) {  
    for(j = 0; j < m; j++) {  
        need[i][j] = max[i][j] - alloc[i][j];  
    }  
}
```

```
int finish[n];  
int safeSeq[n];  
int work[m];
```

```
// Initialize  
for(i = 0; i < n; i++) {  
    finish[i] = 0;  
}
```

```
for(i = 0; i < m; i++) {  
    work[i] = avail[i];  
}
```

```
int count = 0;  
while(count < n) {  
    int found = 0;
```

```

for(i = 0; i < n; i++) {

    if(finish[i] == 0) {

        for(j = 0; j < m; j++) {
            if(need[i][j] > work[j])
                break;
        }
        if(j == m) {

            for(k = 0; k < m; k++) {
                work[k] += alloc[i][k];
            }

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }

    if(found == 0) {
        break;
    }
}

if(count == n) {
    printf("\nSystem is in a safe state.\n");

    printf("Safe sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
    }
}

```

```

        if(i != n - 1)
            printf(" -> ");
    }
}
else {
    printf("\nSystem is NOT in a safe state.\n");
}

return 0;
}

```

```

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)   execution time : 65.397 s
Press any key to continue.
|

```

2) Deadlock Detection

```
#include <stdio.h>
```

```
int main() {
```

```

int n, m;

int i, j, k;

printf("Enter the number of processes: ");
scanf("%d", &n);


printf("Enter the number of resources: ");
scanf("%d", &m);


int alloc[n][m], request[n][m];

int avail[m];

printf("Enter the allocation matrix:\n");
for(i = 0; i < n; i++) {
    printf("Process %d: ", i);
    for(j = 0; j < m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

printf("Enter the request matrix:\n");
for(i = 0; i < n; i++) {
    printf("Process %d: ", i);
    for(j = 0; j < m; j++) {
        scanf("%d", &request[i][j]);
    }
}

printf("Enter the available resources: ");
for(i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}


int finish[n], safeSeq[n];

int work[m];

```

```
for(i = 0; i < m; i++) {  
    work[i] = avail[i];  
}
```

```
// Initialize finish  
for(i = 0; i < n; i++) {  
    finish[i] = 0;  
}
```

```
int count = 0;  
while(count < n) {  
    int found = 0;
```

```
    for(i = 0; i < n; i++) {
```

```
        if(finish[i] == 0) {
```

```
            for(j = 0; j < m; j++) {  
                if(request[i][j] > work[j])  
                    break;  
            }
```

```
            // Process can execute
```

```
            if(j == m) {
```

```
                for(k = 0; k < m; k++) {  
                    work[k] += alloc[i][k];  
                }
```

```
                safeSeq[count++] = i;
```

```

        finish[i] = 1;

        found = 1;
    }
}

if(found == 0)
    break;
}

if(count == n) {
    printf("\nSystem is in safe state.\n");

    printf("Safe Sequence is: ");
    for(i = 0; i < n; i++) {
        printf("P%d ", safeSeq[i]);
    }
}

else {
    printf("\nDeadlock detected!\n");

    printf("Processes in deadlock: ");
    for(i = 0; i < n; i++) {
        if(finish[i] == 0)
            printf("P%d ", i);
    }
}

return 0;

```

}

Enter the number of processes: 5

Enter the number of resources: 3

Enter the allocation matrix:

Process 0: 0 1 0

Process 1: 2 0 0

Process 2: 3 0 3

Process 3: 2 1 1

Process 4: 0 0 2

Enter the request matrix:

Process 0: 0 0 0

Process 1: 2 0 2

Process 2: 0 0 0

Process 3: 1 0 0

Process 4: 0 0 2

Enter the available resources: 0 0 0

System is in safe state.

Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0) execution time : 52.084 s

Press any key to continue.

|